Practical 6

In [12]:

```
import numpy as np
import random
from collections import defaultdict

# ==============================
# Parameters
# ==============================
GRID_SIZE = 4
ACTIONS = ['U', 'D', 'L', 'R']
ACTION_IDX = {a: i for i, a in enumerate(ACTIONS)}

TERMINAL_STATES = [(0, 0), (3, 3)]
START_STATE = (0, 1)

ALPHA = 0.1      # Learning rate
GAMMA = 1.0      # Discount factor
EPSILON = 0.1    # Exploration rate
EPISODES = 50000

# ==============================
# Environment
# ==============================
def step(state, action):
    i, j = state

    if state in TERMINAL_STATES:
        return state, 0, True

    if action == 'U':
        i = max(i - 1, 0)
    elif action == 'D':
        i = min(i + 1, GRID_SIZE - 1)
    elif action == 'L':
        j = max(j - 1, 0)
    elif action == 'R':
        j = min(j + 1, GRID_SIZE - 1)

    next_state = (i, j)
    reward = -1
    done = next_state in TERMINAL_STATES

    return next_state, reward, done

# ==============================
# Epsilon-Greedy Policy
# ==============================
def epsilon_greedy(Q, state):
    if random.random() < EPSILON:
        return random.randint(0, len(ACTIONS) - 1)
    return np.argmax(Q[state])

# ==============================
# Initialize Q
# ==============================
Q = defaultdict(lambda: np.zeros(len(ACTIONS)))

# ==============================
# SARSA Training
# ==============================
for episode in range(EPISODES):

    state = START_STATE
    action_idx = epsilon_greedy(Q, state)

    while True:
        action = ACTIONS[action_idx]
```

```
        next_state, reward, done = step(state, action)

        if done:
            # Terminal update
            Q[state][action_idx] += ALPHA * (reward - Q[state][action_idx])
            break

        next_action_idx = epsilon_greedy(Q, next_state)

        # SARSA update rule
        Q[state][action_idx] += ALPHA * (
            reward + GAMMA * Q[next_state][next_action_idx] - Q[state][action_idx]
        )

        state = next_state
        action_idx = next_action_idx

# ==============================
# Output
# ==============================
print("SARSA training completed.\n")

print("Learned Policy:\n")
policy_symbols = ['↑', '↓', '←', '→']

for i in range(GRID_SIZE):
    row = []
    for j in range(GRID_SIZE):
        state = (i, j)
        if state in TERMINAL_STATES:
            row.append(' T ')
        else:
            best_action = np.argmax(Q[state])
            row.append(f" {policy_symbols[best_action]} ")
    print(" ".join(row))

print("\nSample Q-values:\n")
for state in [(0,1), (1,1), (2,2)]:
    print(f"{state}: {Q[state]}")
```

```
SARSA training completed.

Learned Policy:

 T   ←   ←   ↓
 ↑   ↑   →   ↓
 ↑   →   →   ↓
 ↑   →   →   T

Sample Q-values:

(0, 1): [-2.09490016 -3.20258956 -1.         -3.28235223]
(1, 1): [-2.16804973 -3.167658   -2.39500502 -3.49347598]
(2, 2): [-1.98864195 -1.93190483 -1.95171667 -1.92058469]
```